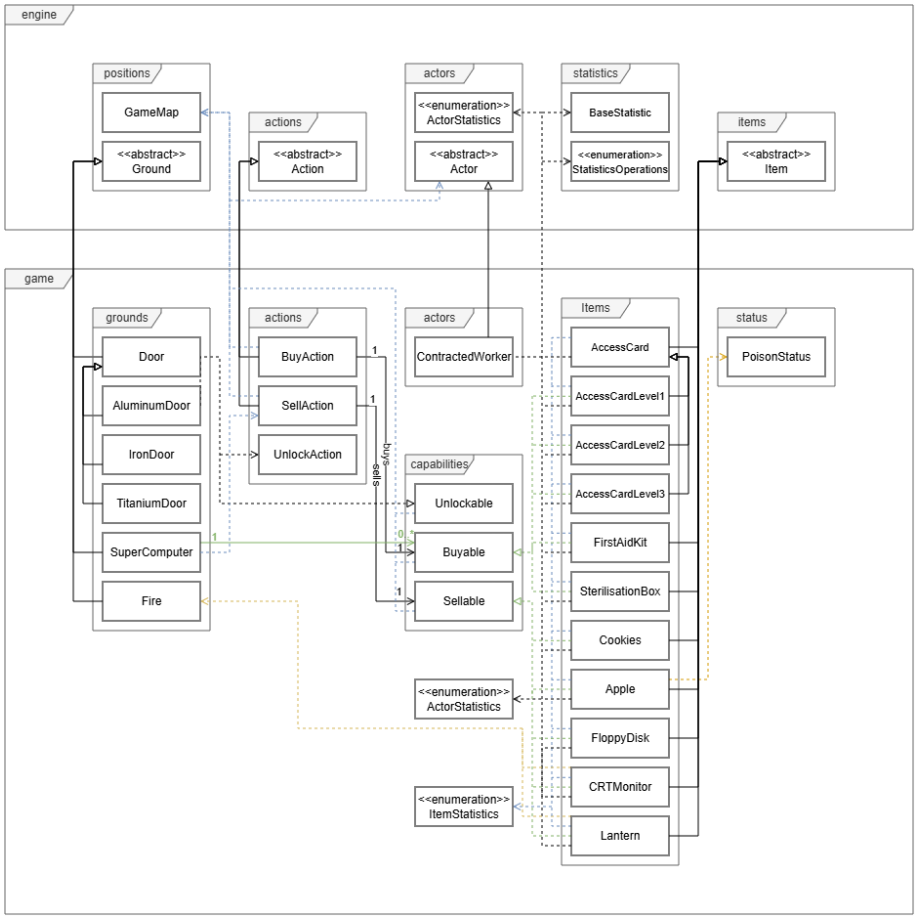
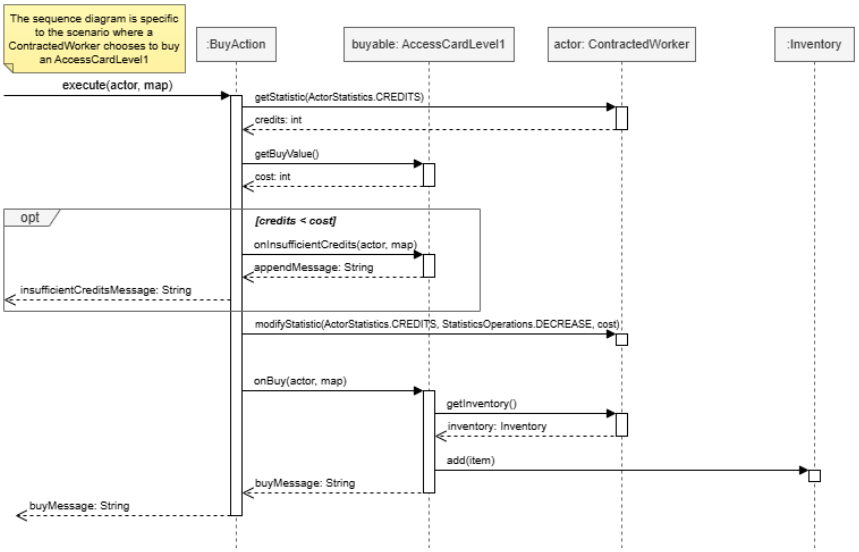
FIT2099 A2 REQ1 Design Document

Design Diagrams

REQ 1 UML Class Diagram:



REQ 1 Sequence Diagram (Buying AccessCardLevel1):



Design Rational

Design 1: Creating an abstract class for Tradeable Items, with their own functionalities written in private methods.

Pros	Cons
By extending the current Item abstract class into another abstract class, we can add functionality needed by the requirement, where if an Item can be traded (Buy/Sell) with attributes indicating the ability, it could be seen as KISS and DRY, as each further extended item would need little addition implementation beyond the new abstract class	If for instance, there were suddenly tradable non-items such as temporary perks or purchasable stats. Suddenly the idea of an abstract class no longer offers any of the features needed. This exposes the idea that an abstract class is rigid in the wrong ways, lacking the robustness object orientation strives for and potentially couples the item system too tightly, creating somewhat of a "God Class".
Connascence is at its lowest here as the method names, parameters and instance variables must match what is provided in the abstract class.	
As most of the code is kept private, exposed public only when needed. This allows for encapsulation.	

Design 2: Creating interfaces for objects that are Buyable and Sellable, with respective Actions extended from Abstract Action

Pros	Cons
It is much easier to scale a project using common interfaces as any implementations of the interfaces will follow somewhat of the same structure but guarantee method implementation when needed.	The Buy and Sell Action class extended from abstract Action in the engine would need a way to access their respective values of the Buyables/Sellables, this does cause repeated code to avoid downcasting but as a result, allows these new Actions to handle all the basic repeated code, such as modification of credit count. This demonstrates DRY but at the cost of an extra method in every class that implements these interfaces.
As the majority of functionality comes from the new Actions, this allows for the Buyable/Sellable objects to not only be limited to Items but any object current and unimplemented. Such as skills which	

could be used in a skill tree or temporary perks, this demonstrates the intent for future use and expandability	
As the main functionality strands from Actions and extended from the abstract Action class, we are reusing code provided in the engine. Following proper coding practice, mitigating potentially unintentional code smells and promoting correct usage of OOP.	

Design 3: Creating concrete classes for each object, one for the Buyable version and another for the Sellable, using an Action extended from the abstract Action class to join the system to the engine

Pros	Cons
Streamlined to implement the exact functionality required	Not utilising provided codebase of which provides abstract classes, interfaces with methods and attributes that would support DRY
KISS is enforced as only what is outlined in the requirement would need to be developed	Almost impossible to expand if new requirements are asked for due to tight coupling from non-OOP programming
	It is very likely that the class would be cluttered with many irrelevant functionalities due to how late into development this requirement already is, breaking the first rule of SOLID, where the class should only be handling the Buying or Selling

Summary

The UML diagram shows the implementation of requirement 1, which follows Design 2.

The aim for this requirement was to avoid abstraction and maintain the interface driven codebase, following the structure already used with Actions such as ConsumeAction and UnlockAction. This allows for the SOLID principles and Connascense of Name to be followed when implementing the Buyable/Sellable interfaces. For instance, in the provided sequence diagram, it demonstrates a ContractedWorker buying an AccessCardLevel1. By structuring the code using the Action abstraction/Interfaces, the only unique implementations are the actual results of buying the item, in which this case simply deposits the item into the backpack, but of which is not at all limited just to.

To compare to the other designs 1 & 3, which take the approach toward abstract classes and further abstraction, I believe they are both fundamentally flawed. They either limit themselves to constantly rewriting and refactoring existing functionalities and lacking the futureproof-ness OOP strives for. As described in the Pros/Cons:

> *"If for instance, there were suddenly tradable non-items such as temporary perks or purchasable stats. Suddenly the idea of an abstract class no longer offers any of the features needed. This exposes the idea that an abstract class is rigid in the wrong ways, lacking the robustness object orientation strives for and potentially couples the item system too tightly, creating somewhat of a "God Class"."*

This is why I firmly consider the strict usage of interfaces as the objectively correct design choice for this requirement, not limiting the Buy/Sell features to just items but any game instance while maintaining single responsibility.

Additionally, I refactored some of the requirements from Assignment 1, focusing on keeping it simple (KISS) and not repeating myself (DRY), this can be seen though the constant use of interfaces opposed to concrete classes to somewhat force structure but allow loose coupling and when needed. For instance, Unlockable & Consumable, of which Buyable/Sellable are based on and follow the exact same structure. Where the interface requires the implementation of a method that gets executed when their respective Action is used, minimising the repetition of logic.